

THE UNIVERSITY
OF BRITISH COLUMBIA

Faculty of Science

Hanson Sun

BSc. (Hons) Computer Science

Depth-Inference Capable and Robust PhysTwin

May 2026

Department of Computer Science

Abstract

PhysTwin is a recent real-to-simulation (real2sim) pipeline that converts RGBD video into a spring–mass parameterised *warp* digital twin, but it assumes that depth and camera calibration are already available.

This project presents DIPhysTwin (Depth-Inference PhysTwin), which builds an interactive digital twin from multi-view RGB video alone. Camera poses and metric scale are recovered with DUSt3R, and dense depth is inferred with Depth Anything 3 (DA3), which predicts spatially consistent geometry from arbitrary image sets. In our fixed-camera videos, however, its depth varies between frames. The available streaming adaptation processes overlapping chunks, but its Umeyama alignment breaks down on larger chunks; duplicating static views helps only over short windows. The central contribution of this work is a lightweight temporal depth smoother that learns a residual correction on top of the DA3 prediction. It needs no ground-truth depth: temporal behaviour is supervised by a video depth model, while fidelity and edge terms preserve the geometry and boundaries of the baseline.

DIPhysTwin is evaluated on PhysTwin's released data and on approximately 30 self-recorded multi-view sessions spanning cloth, mats, bottles, mugs and stuffed animals. It produces working warp models with pose, depth and ground alignment all inferred rather than measured, and the quality of the result depends far more on capture conditions than on any single stage: large, slow-moving objects with little controller occlusion reconstruct well, whereas small, fast-moving objects lose 3D consistency. The remaining limitations, including the absence of true multi-view streaming inference, imperfect capture and calibration, and the lack of a robust 3D point tracker, are analysed and used to outline future work.

Contents

1 Introduction **1**

1.1 Motivation and Problem statement 1

1.2 Contributions 1

2 Background and Related Work **3**

2.1 PhysTwin and Real2Sim Digital Twins 3

2.2 Monocular and Video Depth Estimation 3

2.3 Camera Pose Estimation 4

2.4 Temporal Depth Stabilisation 4

2.5 Dense Tracking and 3D Reconstruction 5

2.6 Summary of the Gap 6

3 System Design **7**

3.1 Pipeline Overview 7

3.2 Data Collection 8

3.3 Pose Estimation and Extrinsics Alignment 9

3.4 Depth Inference 9

3.5 Segmentation, Shape Prior and Dense Tracking 10

3.6 Implementation and Robustness Fixes 10

4 Temporal Depth Smoother **12**

4.1 Motivation 12

4.2 Learning a Residual Correction 12

4.3 Architecture 13

4.4 Loss Functions 16

4.5 Training . . . 18
4.6 Inference . . . 18
4.7 Properties and Limitations . . . 20

5 Evaluation **22**
5.1 Evaluation Setup and Datasets . . . 22
5.2 Measuring Depth Quality Without Ground Truth . . . 22
5.3 Depth against the sensor reference . . . 23
5.4 Flicker removal across sequences . . . 25
5.5 Effect on Tracking . . . 27
5.6 End-to-End Results . . . 28
5.7 Comparison with the original pipeline . . . 29
5.8 Findings and Failure Cases . . . 31
5.9 Negative Results . . . 32

6 Discussion, Limitations and Future Work **33**
6.1 Summary and Diagnosis . . . 33
6.2 Limitations . . . 34
6.3 Future Work . . . 34
6.4 Conclusion . . . 35

References **36**

A Extended Results and Result Videos **39**
A.1 Improvement Chain on the Sloth Sequence . . . 39
A.2 Effect of the Temporal Depth Smoother . . . 40
A.3 Depth-Source Variants . . . 40
A.4 Streaming Inference . . . 40
A.5 Tracking Comparison with SpatialTrackerV2 . . . 41
A.6 Environment Interaction: Rope with and without Ground Alignment . . . 41
A.7 Individual Demonstrations . . . 41

B Hyper-parameters and Loss Weights **43**

C Source Code **44**

Chapter 1

Introduction

1.1 Motivation and Problem statement

Policies for deformable manipulation are learned from recordings of real interactions, and digitising those recordings by hand takes 3D modelling, material parameters and manual matching. The goal is to automate that process, converting multiple RGB video recordings of the same interaction into a single *digital twin* of the manipulated object.

Jiang et al. (2025) proposed PhysTwin, a real2sim pipeline that takes RGBD video from several cameras, segments the object, tracks the object and the controller, and fits a spring–mass *warp* model for interaction in NVIDIA Warp (Macklin 2022). This work improves the robustness of PhysTwin by (1) removing the depth sensor, (2) removing camera calibration and (3) tolerating imperfect segmentation and tracking; the resulting pipeline is called DIPhysTwin. It should allow PhysTwin to be used in more flexible capture conditions, with varying degrees of camera setup and less human intervention.

1.2 Contributions

The main contributions of this work are:

1. **A depth-inference-first pipeline.** Poses and metric scale come from a dedicated stage rather than the depth model, and depth is estimated per view.
2. **A ground-truth-free temporal depth smoother.** A small network learns a *residual correction* to one depth model's maps, smoothing temporal flicker in depth videos while

preserving spatial consistency.

3. **Robustness engineering.** The pipeline handles resolutions independently, runs idempotently, aligns the scene to the ground with RANSAC and propagates that transform to poses, tracks and meshes, with fallbacks when shape prior fitting or dense tracking fails.

Chapter 2

Background and Related Work

This chapter reviews the core components referenced in the report and identifies the gaps this project addresses.

2.1 PhysTwin and Real2Sim Digital Twins

PhysTwin (Jiang et al. 2025) takes synchronised multi-view RGBD video of a controller interacting with an object, segments the object, and tracks dense motion on both the object and the controller. It lifts those tracks into 3D with depth and trains a spring–mass *warp* model in NVIDIA Warp (Macklin 2022). The trained model responds to a controller point in the simulator and reproduces soft-body behaviour.

The PhysTwin pipeline assumes cameras fixed in space while the object and controller(s) move. This assumption complicates multi-view depth reconstruction.

2.2 Monocular and Video Depth Estimation

VDA (Chen et al. 2025) is a monocular depth model that produces temporally consistent maps for a single view. Depth Anything 3 (DA3) (Lin et al. 2026) predicts spatially consistent geometry from an arbitrary number of images, with or without known camera poses. Multi-view input does not by itself ensure temporal consistency for a moving scene recorded by stationary cameras. In the configuration tested here, depth varies between frames. The streaming adaptation processes overlapping chunks, but its Umeyama alignment becomes singular for larger

chunks; duplicating the static views as a dynamic sequence helps only over a limited window (Chapter 3).

DUSt3R and MASt3R (S. Wang et al. 2024; Leroy et al. 2024) use pairwise 3D correspondences to recover poses. Their estimates also vary over time in the tested sequences.

As of the current state of research, no model is both spatially and temporally correct. Models that reason over one frame or several views flicker between frames, while video-consistency models are smooth but geometrically weaker and monocular (Chapter 4).

2.3 Camera Pose Estimation

Intrinsics and extrinsics serve two purposes here: aligning depth maps with the RGB frames used for segmentation, and triangulating tracked 2D points into 3D tracks. They are not always measured during capture.

Pose from the depth model was poor in both respects, misaligning 3D projections and degrading optimisation and training. Empirically, pairwise correspondences from DUSt3R with a heuristic scale aligned much better (Chapter 3).

2.4 Temporal Depth Stabilisation

VDA (Chen et al. 2025) adds temporal attention at several decoder scales and introduces a temporal gradient matching (TGM) loss. That loss supervises frame-to-frame *change* in depth rather than absolute depth, and inference runs over a segment-wise sliding window.

StableDPT (Sobko et al. 2026) attends spatially within each frame in the encoder and temporally across frames in the decoder. It also adopts the TGM loss.

NVDS and NVDS+ (Yiran Wang, Shi, J. Li, Huang, et al. 2023; Yiran Wang, Shi, J. Li, Hong, et al. 2025) stabilise depth after the fact, using sliding-window cross-attention together with an optical flow model, but they require metric depth.

Table 2.1: Comparison of prior temporal depth stabilisation methods

Method	Temporal mechanism	Supervision	Ground-truth depth	Multi-camera input
VDA	temporal attention at multiple decoder scales, segment-wise sliding window	TGM loss	yes (lidar)	no (monocular video)
StableDPT	spatial attention per frame, temporal attention across frames in decoder	TGM loss	yes (lidar)	no (monocular video)
NVDS / NVDS+	post-hoc stabiliser, sliding-window cross-attention + optical flow	temporal loss via flow model	yes (sensor ground truth)	no (monocular video)
Li et al. (2023)	not stated	not stated	not stated	not verified

All four methods need ground-truth or metric depth, and all four are monocular: each takes a single video stream and none accepts synchronised views from several cameras. This project instead needs a smoother for several synchronised views from an uncalibrated capture, trained *without ground-truth depth* by treating a video depth model (VDA) as a motion oracle.

2.5 Dense Tracking and 3D Reconstruction

CoTracker (Karaev et al. 2024) tracks points jointly rather than independently, which makes it more stable than pairwise optical flow. It is the tracker the base pipeline uses, and since it works in 2D, its tracks are lifted into 3D with the estimated depth. Tracking accuracy is therefore bounded by depth accuracy.

MVTracker (Rajič et al. 2025) tracks across several views, SpatialTrackerV2 (Xiao et al. 2025) tracks in 3D. On the captures used here, the multi-view tracker performed worse than CoTracker and the 3D tracker produced visible artefacts (Chapter 5).

SAM and its successors (Kirillov et al. 2023; AI 2025; Ren et al. 2024) produce promptable, open-vocabulary masks, and their quality depends strongly on how the object is described. Trellis (Xiang et al. 2025) was evaluated as a source of mesh and shape priors, while appearance is reconstructed with Gaussian splatting (Kerbl et al. 2023).

2.6 Summary of the Gap

Each area above is well developed on its own, but few models combine the three properties needed here. No widely used model is spatially consistent, temporally smooth and multi-camera capable at once. Existing stabilisers need ground-truth depth and are monocular, and most multi-camera reconstruction assumes a stationary scene with a moving camera, whereas this project observes a moving object from static cameras.

DIPhysTwin addresses the gap in two ways. The first is a ground-truth-free smoother that learns a residual correction to flickering depth, with a video model serving only as a motion oracle (Chapter 4). The second is a set of robustness measures that let the pipeline tolerate imperfect input (Chapter 3).

Chapter 3

System Design

3.1 Pipeline Overview

DIPhysTwin modifies the data-processing and generation stages of PhysTwin; warp training is unchanged. The new and modified stages are:

1. **DUSt3R pose estimation with heuristic scaling** (new). Camera intrinsics, extrinsics and a metric scale for the scene are recovered here.
2. **DA3 depth inference** (new). Dense depth is estimated for every frame of every view with Lin et al. (2026).
3. **Temporal depth smoothing** (new). A small network learns a residual correction to the DA3 depth maps, which reduces temporal flicker while preserving spatial consistency.
4. **Data processing** (modified PhysTwin). Depth maps, masks and images are combined into the per-frame structure the pipeline expects.
5. **Extrinsics alignment** (new). The scene is rotated until the ground plane is flat, with everything above it.

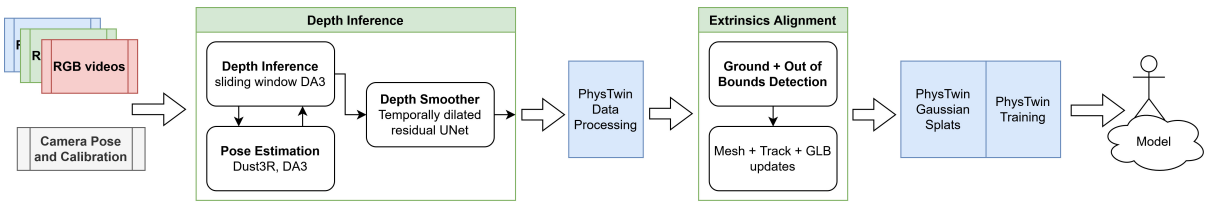


Figure 3.1: The DIPhysTwin pipeline: camera calibration and pose estimation feed depth inference, whose smoothed depth maps drive the modified PhysTwin stages.

3.2 Data Collection

Three datasets are used. The first is the data released with PhysTwin (Jiang et al. 2025), captured with a depth sensor; the second was collected for this project with a looser protocol, and the third with a stricter one. The first set is used to verify that the pipeline can reproduce PhysTwin's results, while the second and third sets are used to evaluate the pipeline's robustness. Table 3.1 summarises the released data.

Table 3.1: The data released with PhysTwin (Jiang et al. 2025), summarised from the dataset description. The sensor depth gives this dataset a depth reference, as does the stricter self-collected set.

Property	Released data
Scenarios	22 recorded interaction scenarios
Object types	Ropes, stuffed animals, cloth, delivery packages
Interactions	Quick lifting, stretching, pushing and squeezing, with one or both hands
Video length	1–10 seconds per interaction
Capture rig	Three RealSense D455 RGBD cameras
Annotations	9 manually annotated ground-truth tracking points per video
Split	7:3 train/test per scenario; only the training split constructs the twin

The second self-collected set contains roughly eight laboratory interactions. The objects span soft materials, such as a mat, stuffed animals and a plush sloth, and hard ones, such as a T-shirt and several bottles. Intrinsics and extrinsics were deliberately not measured, because the cameras differed from one another and the pipeline was expected to infer them. Figure 3.2 shows examples from both protocols. Data was collected with weaker constraints (two phones and a DJI action camera), rolling shutters, automatic exposure and focus, and no manual alignment of start times. This set exposed the capture problems discussed in Chapter 5.

The third set follows a stricter protocol and contains roughly thirty sessionscovering cloth, mats, bottles, mugs and stuffed animals. Three Intel RealSense D415 cameras recorded colour and depth together, so these sessions carry a depth reference; the intrinsics and extrinsics were measured manually as a reference only, since the pipeline used inferred poses; start times were aligned in software. Chapter 6 treats these properties as a source of error.

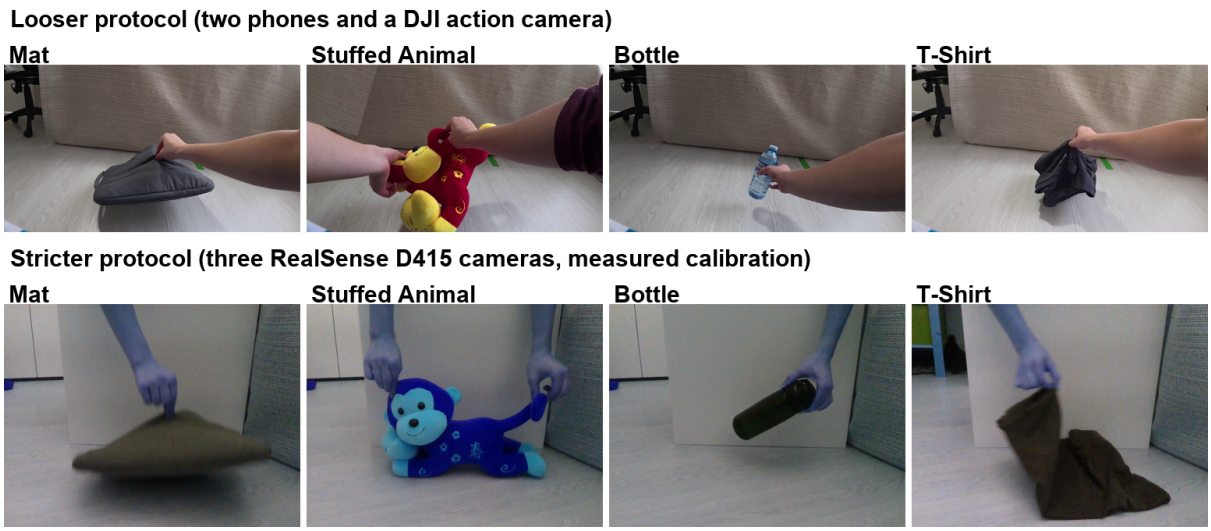


Figure 3.2: Example frames from the self-collected sessions: the same object categories captured under the looser protocol (top) and the stricter protocol (bottom).

3.3 Pose Estimation and Extrinsics Alignment

Pose estimation is a separate stage: S. Wang et al. (2024) builds pairwise 3D correspondences that align much better, with a heuristic for metric scale (scaling the camera separation to 1 m), and DA3's own pose estimation is less accurate. The estimated parameters feed data processing and alignment.

3.4 Depth Inference

Depth is estimated with Lin et al. (2026), which produces non-metric but geometrically consistent views given several camera poses. Those poses may describe a dynamic scene, or cameras positioned around a static one.

In the inference configuration used here, depth varies between time steps and boundaries are inconsistent. This weakens the reconstruction and adds noise to the training data. To improve consistency, the pipeline duplicates views from stationary cameras and presents them as a dynamic sequence, a "fake" sliding window. This helps over a limited number of frames. Larger windows make the Umeyama alignment (Umeyama 1991) singular, so this procedure does not adapt reliably to the sequences used here. An attempt to adapt the full model to accept multiple poses produced worse results and was abandoned.

Depth and pose estimation leave the scene in an arbitrary frame: the floor can be tilted, or the

object can sit below or above it, but the simulator needs the ground at $z = 0$.

The current stage back-projects the depth maps into world coordinates and fits the dominant plane with RANSAC. The fitted plane is rotated onto $z = 0$, and the same rigid transform is applied to camera poses, tracked points and meshes so that everything stays in one frame. The alignment script takes the first fitted plane as the floor without checking it, and leaves a downward-pointing normal as it is.

This gives the simulator a ground plane at $z = 0$. Supported objects were no longer over-stiff once ground contact was represented.

3.5 Segmentation, Shape Prior and Dense Tracking

Segmentation uses a promptable model (Kirillov et al. 2023). More descriptive object names produce better masks, but some sessions still need manual intervention. Two failures were then addressed downstream:

- **Split masks.** Occlusion by the controller can break the mask into disjoint regions, which defeats dense tracking. Morphological operations first rejoin them.
- **Non-watertight meshes.** The as-rigid-as-possible shape prior alignment (Sorkine and Alexa 2007) fails on the meshes these reconstructions produce. The fallback applies the average translation of the tracked points.

3.6 Implementation and Robustness Fixes

Several fixes cut across stages; Figure 3.3 marks where they apply.

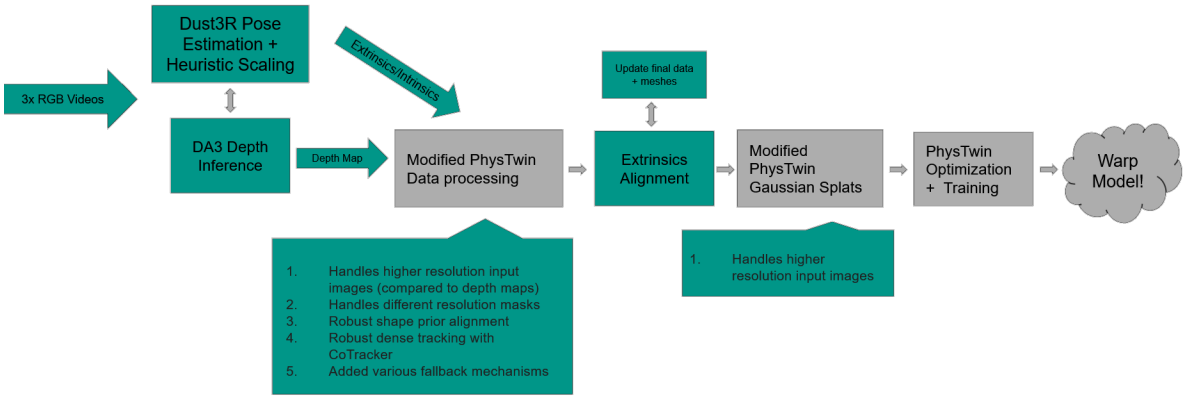


Figure 3.3: The DIPhysTwin stages with the robustness fixes marked.

- **Flexible resolutions.** Hardcoded image, video and depth resolutions were incompatible with the collected data, which records video at 1080p while depth maps are far smaller at 280×504, and with components that prefer higher resolution. Video, depth maps and masks now take independent resolutions, converted explicitly.
- **Idempotency.** Repeated runs produced different results, because resolution changes and intrinsics were applied more than once, misaligning objects and spoiling the gaussian data. Repeated runs now agree.
- **Fallbacks.** Where a stage can fail outright, the pipeline continues rather than stopping, as in the shape prior alignment described above.
- **Assumptions removed.** The pipeline assumed particular position ranges within the frame and a particular attachment for the controller point. Both have been generalised.

Chapter 4

Temporal Depth Smoother

A learned post-processing stage between depth inference and data processing removes temporal flicker from inferred depth.

4.1 Motivation

Depth inference (Chapter 3) runs a geometry model per frame on "fake" sliding-window input. Its depth is temporally unstable. Surfaces that do not move change depth from one frame to the next, and the object–background and object–controller boundaries shift, so the silhouette moves while the object itself is still. Since 2D correspondences are lifted into 3D with these values, that jitter reaches the tracked points, the reconstruction, and the warp model.

Video depth models such as Chen et al. (2025) are smooth, but they are geometrically weak and monocular, so they do not suit a multi-camera capture (see the results comparison in Table A.4, Appendix A). Earlier stabilisation work either couples to a specific depth backbone (Yiran Wang, Shi, J. Li, Huang, et al. 2023; Yiran Wang, Shi, J. Li, Hong, et al. 2025) or assumes labelled sensor ground truth (Sobko et al. 2026; P. Li et al. 2023). The stage described here has to meet a narrower requirement: it must apply to any flickering depth sequence, need no ground truth, and work across views.

4.2 Learning a Residual Correction

The two model families fail in opposite places: the geometry model has good *spatial* structure, and the video model has good *temporal* structure. Depth and RGB are encoded into a latent

space and decoded into a smooth prediction, which is scored by temporal penalties taken from the video model and geometric penalties taken from the geometry model.

Rather than predict a depth map, the network predicts a *residual* that is added to the geometry model's prediction. The video model serves as a motion oracle, not a depth reference: its values are relative, so no metric scale is inherited from it.

4.3 Architecture

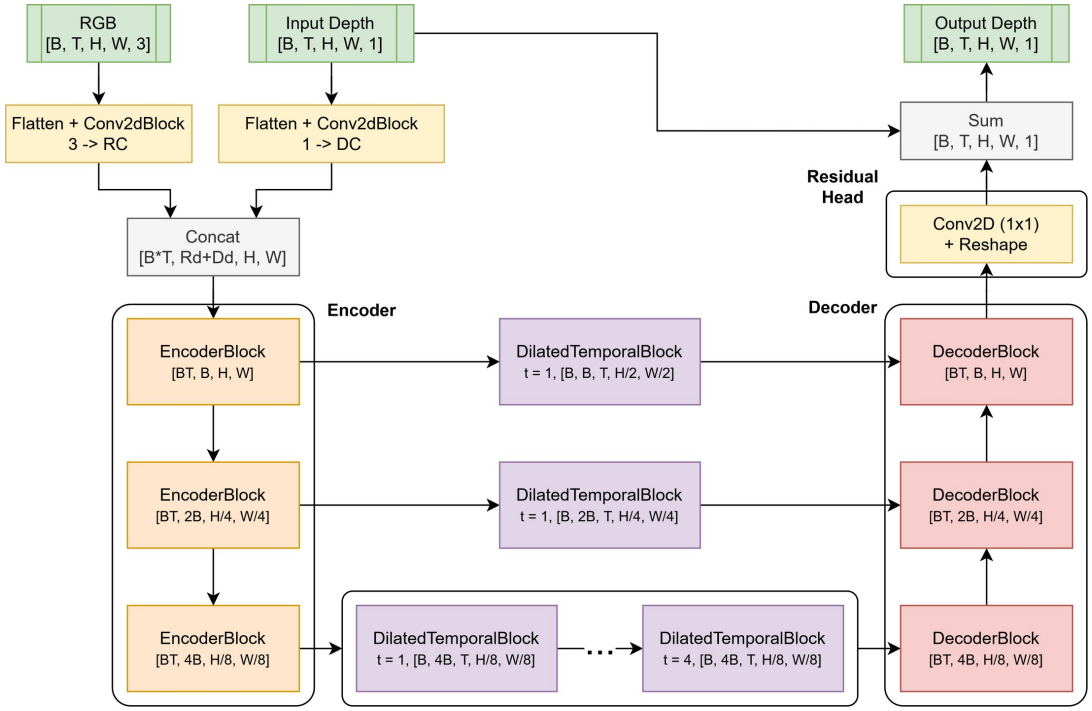


Figure 4.1: The temporally-dilated residual U-Net.

The network has to learn and preserve structure from the geometry model, absorb temporal consistency from the video model, and correct each frame using both depth maps and the RGB image. A diffusion- or DPT-style backbone was considered and rejected, because it was deemed overkill and potentially too expensive. The chosen design is a small 4.7-million-parameter **temporally-dilated residual U-Net**. Spatial structure is preserved across scales, and temporal dilation gives each frame context from several time offsets without processing the whole clip. Its size is limited by the training compute available.

The U-Net is assembled from four blocks (Figure 4.2). The per-frame 2D block, `Conv2dBlock`, is two 3×3 convolutions, each followed by GroupNorm and ReLU. An `EncoderBlock` runs one of these and taps the skip connection before a strided 2×2 convolution halves the resolution. A

`DecoderBlock` doubles the resolution with a transposed convolution, concatenates the matching skip and refines the result with a `Conv2dBlock`; a learned upsample is used because bilinear upsampling produced smearing and edge artefacts. The `DilatedTemporalBlock` is a single $3 \times 3 \times 3$ convolution, dilated only in time, with GroupNorm, ReLU and a residual connection. Stacking dilations $1, 2, 4, 8$ at the bottleneck gives each output frame a receptive field of ± 15 frames; shallower levels use dilation 1.

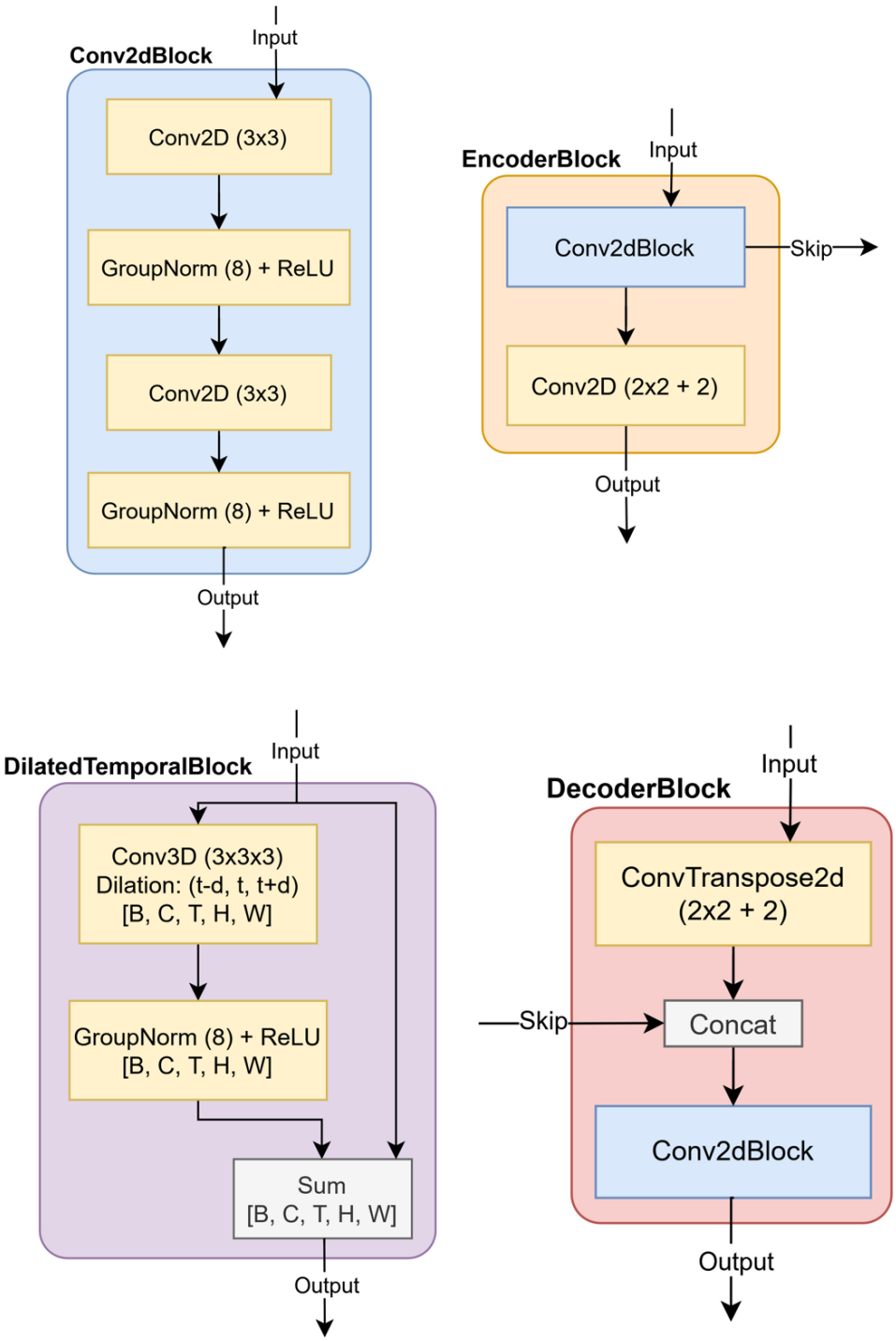


Figure 4.2: The four building blocks of the smoother, drawn to a common scale: (a) `Conv2dBlock`, (b) `EncoderBlock`, (c) `DilatedTemporalBlock` and (d) `DecoderBlock`, which reinserts the skip connection that the encoder taps.

4.4 Loss Functions

The objective combines fidelity, temporal gradient matching (TGM), temporal total variation (TV), and two geometric penalties: edge sharpness and spatial-gradient matching.

Let D^s, D^0 and D^v denote the smoothed output, raw DA3 depth and aligned VDA depth. For frame skip k, define

$$\Delta_k D_{b,t,x} = D_{b,t+k,x} - D_{b,t,x}, \qquad \Delta_k^2 D_{b,t,x} = D_{b,t+2k,x} - 2D_{b,t+k,x} + D_{b,t,x}.$$

Here b indexes a clip, t a frame and x a pixel. Angle brackets denote a mean over valid clips, frame indices and pixels. Denominators are clamped below by $\epsilon = 10^{-6}$. The equations follow the implementation.

Temporal gradient matching loss. For each span k, TGM matches temporal depth differences to VDA. The VDA values provide a motion signal, not a depth target. Each span is normalised by the discrepancy between raw DA3 and VDA:

$$B_{\text{tgm}}^{(k)} = \max\left(\left\langle \left|\Delta_k D^0 - \Delta_k D^v\right| \right\rangle, \epsilon\right), \qquad \mathcal{L}_{\text{tgm}}^{(k)} = \frac{\langle |\Delta_k D^s - \Delta_k D^v| \rangle}{B_{\text{tgm}}^{(k)}}.$$

The implementation combines $k = 1, 2, 3$ as

$$\mathcal{L}_{\text{tgm}} = \mathcal{L}_{\text{tgm}}^{(1)} + 0.25\, \mathcal{L}_{\text{tgm}}^{(2)} + 0.07\, \mathcal{L}_{\text{tgm}}^{(3)}.$$

Larger skips capture slower oscillations; their weights are code defaults and have not been systematically tuned.

Temporal total variation loss. TV penalises the output's second temporal difference. VDA acceleration gates the penalty, reducing it where the motion oracle reports acceleration. For each span,

$$w_{b,t,x}^{(k)} = \exp\left(-10 \left|\Delta_k^2 D_{b,t,x}^v\right|\right), \qquad B_{\text{tv}}^{(k)} = \max\left(\left\langle \left|\Delta_k^2 D^0\right| w^{(k)} \right\rangle, \epsilon\right),$$

$$\mathcal{L}_{\text{tv}}^{(k)} = \frac{\left\langle \left|\Delta_k^2 D^s\right| w^{(k)} \right\rangle}{B_{\text{tv}}^{(k)}}, \qquad \mathcal{L}_{\text{tv}} = \mathcal{L}_{\text{tv}}^{(1)} + 0.25\, \mathcal{L}_{\text{tv}}^{(2)}.$$

The exponential constant and span weights come from the implementation and were not tuned.

Fidelity loss. Fidelity keeps the output close to DA3 in a shared, clip-level scale. Let $m_{b,x} = \operatorname{median}_t D^0_{b,t,x}$, $t_b = \operatorname{median}_x m_{b,x}$ and $s_b = \max(\operatorname{mean}_x |m_{b,x} - t_b|, \epsilon)$. The aligned depth is $A_b(D) = (D - t_b)/s_b$. For the VDA gate, let $\mu_v = \langle |\Delta_1 D^v| \rangle + \epsilon$ and $\gamma = 0.25$:

$$g_{b,0,x} = 1, \qquad g_{b,t,x} = \gamma + (1-\gamma) \exp\left(- \frac{|\Delta_1 D^v_{b,t-1,x}|}{\mu_v} \right), \quad t > 0,$$

$$B_{\text{fid}} = \max\big(\operatorname{std}(A(D^0)), \epsilon\big), \qquad \mathcal{L}_{\text{fid}} = \frac{\big\langle g \, |A(D^s) - A(D^0)| \big\rangle}{B_{\text{fid}}}.$$

The gate is strongest in temporally stable regions and relaxes toward 0.25 where VDA reports motion. Its scale and location statistics are computed from DA3, not VDA.

Geometric loss. The implementation has two spatial penalties. Define forward finite differences by $\nabla_x D_{b,t,i,j} = D_{b,t,i,j+1} - D_{b,t,i,j}$ and $\nabla_y D_{b,t,i,j} = D_{b,t,i+1,j} - D_{b,t,i,j}$, padding the last column and row with zero. The edge-sharpness penalty uses Sobel magnitudes $G(I)$ of grayscale RGB and $G(D^v)$ of VDA depth, each divided by its per-clip 95th percentile. It keeps pixels identified as edges in both signals and penalises smoothed edges below 0.8 of the VDA edge magnitude:

$$M = \mathbf{1}[\widetilde{G}(I) > 0.6] \, \mathbf{1}[\widetilde{G}(D^v) > 0.6], \qquad \mathcal{L}_{\text{edge}} = \frac{\operatorname{mean}_{M=1} [\, 0.8 \widetilde{G}(D^v) - \widetilde{G}(D^s) \,]_+}{\max\Big(\operatorname{mean}_{M=1} \widetilde{G}(D^v), \epsilon \Big)}.$$

Here $[q]_+ = \max(q, 0)$; an empty mask gives zero numerator. The second penalty compares forward finite-difference gradients of smoothed and raw DA3 depth, weighted by VDA Sobel magnitude. With $W = \max(0.1, G(D^v)/(\operatorname{mean}_{t,x} G(D^v) + \epsilon))$,

$$\mathcal{L}_{\text{grad}} = \frac{\frac{1}{2} \sum_{a \in \{x,y\}} \big\langle W \, |\nabla_a D^s - \nabla_a D^0| \big\rangle}{\max\Big(\frac{1}{2} \sum_{a \in \{x,y\}} \langle W |\nabla_a D^0| \rangle \, , \epsilon \Big)}.$$

Both spatial terms are baseline-normalised in the implementation. Their individual effects have not been isolated in the evaluation.

Combined objective. The weighted objective is

$$\mathcal{L} = \lambda_{\text{fid}} \mathcal{L}_{\text{fid}} + \lambda_{\text{tgm}} \mathcal{L}_{\text{tgm}} + \lambda_{\text{tv}} \mathcal{L}_{\text{tv}} + \lambda_{\text{edge}} \mathcal{L}_{\text{edge}} + \lambda_{\text{grad}} \mathcal{L}_{\text{grad}}.$$

The training configuration gives $\lambda_{\text{fid}} = 1.3$, $\lambda_{\text{tgm}} = 0.8$, $\lambda_{\text{tv}} = 0.8$, $\lambda_{\text{edge}} = 0.05$ and $\lambda_{\text{grad}} = 0.04$. The span weights are listed in Appendix B. These settings remain untuned.

4.5 Training

The training run uses 162 complete clip-camera inputs and saves a best-model checkpoint. The deterministic split is by clip, so frames from a validation clip do not enter training. Table 4.1 records the training configuration.

Table 4.1: Temporal smoother training configuration.

Setting	Recorded value
Complete clips	162 clip-camera inputs; 130 train and 32 validation (80:20, seed 42, split by clip)
Training samples	1,842 windows; length 16 frames, stride 8
Validation samples	264 windows; length 16 frames, stride 16
Input size	226×334 pixels; RGB and depth are resized
Batch and workers	Batch size 2; 2 data-loader workers
Optimiser	Adam; weight decay 10^{-4}
Learning rate	3×10^{-3}; ReduceLROnPlateau, factor 0.5, patience 5
Duration	50 epochs; checkpoints saved for each epoch
Precision	CUDA autocast with bfloat16 enabled
Loss weights and temporal spans	See Table B.2 (Appendix B)
Augmentation and hardware	No augmentation in the loader; NVIDIA GeForce RTX 3060

Training ran on a single NVIDIA GeForce RTX 3060. The loader resizes and normalises inputs but applies no augmentation.

4.6 Inference

Inference follows the video model. Clips are processed in overlapping chunks, which provides temporal context without holding an entire sequence in memory; the chunks are blended with the weighted window of Figure 4.3. Resolutions are arbitrary with minimal padding, and depth maps are never resized, since resizing would destroy the sub-pixel frame-to-frame consistency the stage exists to produce.

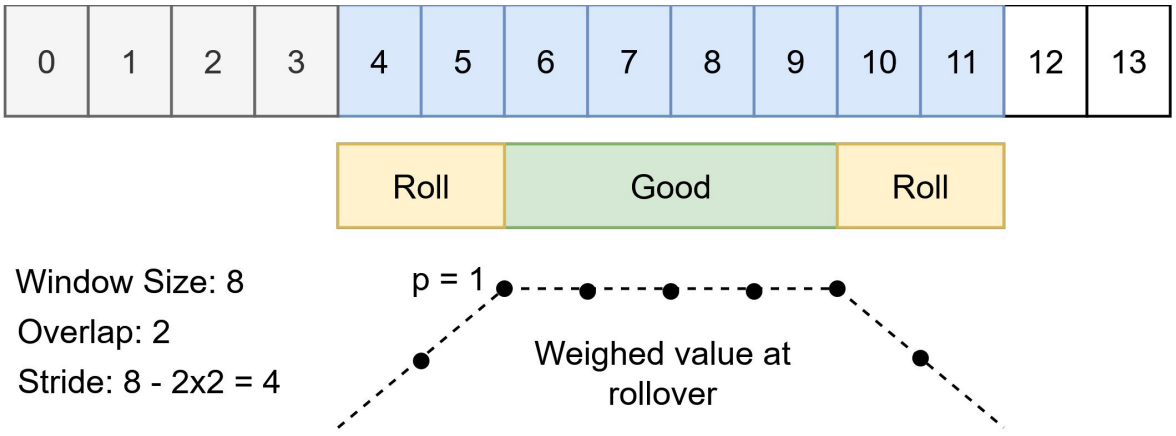


Figure 4.3: Weighted sliding-window inference. Overlapping chunks are blended into one output sequence.

Figure 4.4 shows four frames, two each from a mat lift and a sloth stretch, before and after smoothing with the best saved model. Consecutive-frame comparisons over five sequences are shown in Figure 5.3 (Chapter 5); full recorded clips are listed in Appendix A.

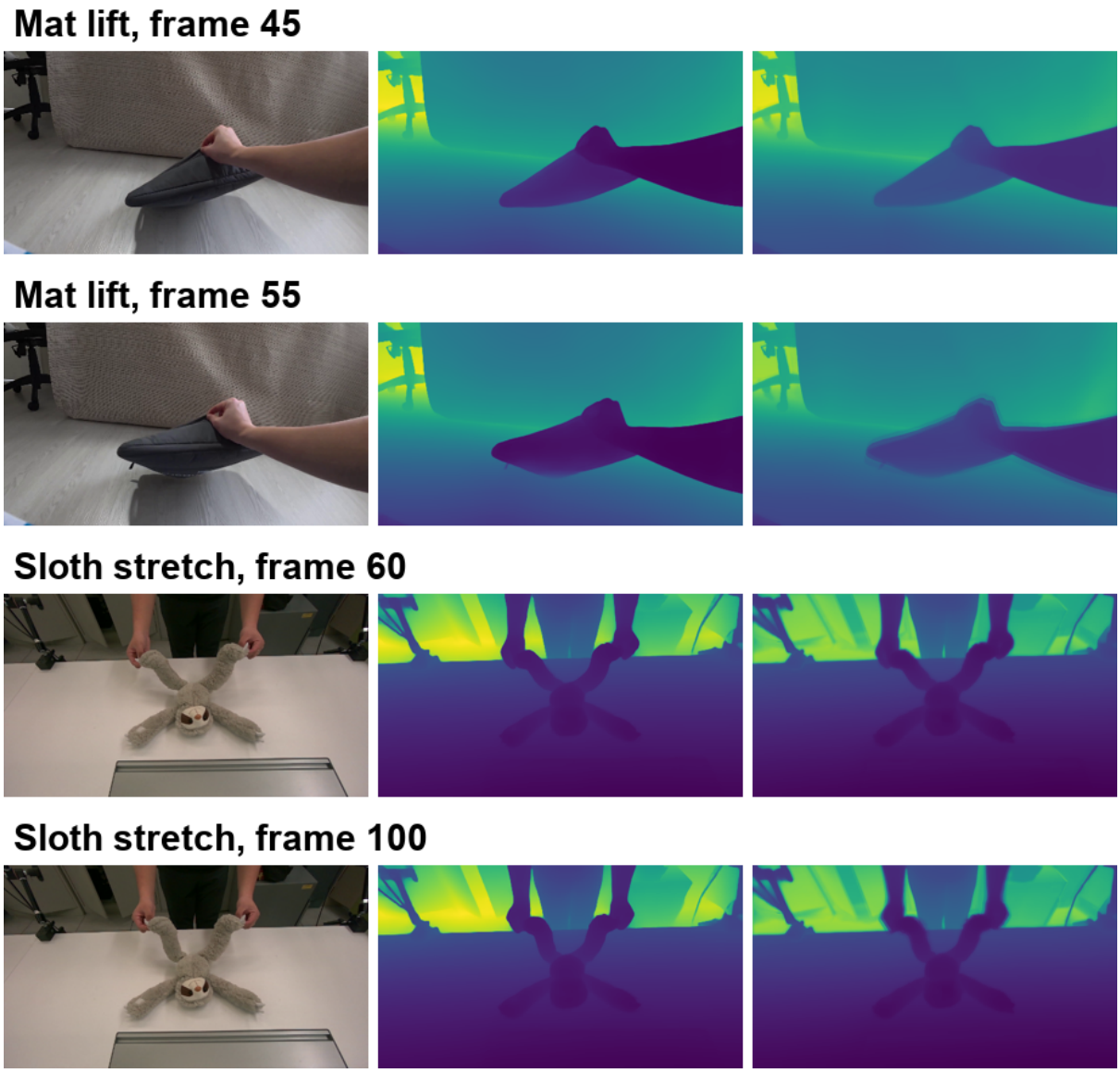


Figure 4.4: RGB input, raw DA3 depth and smoothed depth for four frames of a mat-lift and a sloth-stretch sequence.

4.7 Properties and Limitations

The stage can be reused independently of this project:

1. It applies to any flickering depth map, because it predicts a correction and needs no specific backbone.
2. It requires no ground-truth depth and no labelled dataset.
3. It handles arbitrary resolutions without resizing.
4. It is small and fast relative to the depth model it corrects.

Some limitations remain. Smoothing visibly blurs depth, and there is not enough evidence that the correction targets object flicker vs stationary flicker. The hyper-parameters are untuned, and the results depend heavily on these settings. The stage has not been validated on other depth backbones, and it is not clear how well it generalises to other domains.

Chapter 5

Evaluation

This chapter reports the depth comparison and qualitative pipeline results. The looser self-collected videos have no depth ground truth, so most measures here address stability and geometry preservation rather than accuracy.

5.1 Evaluation Setup and Datasets

The evaluation uses PhysTwin's released RGBD data (Jiang et al. 2025) and about 30 self-collected multi-view sessions of cloth, mats, bottles, mugs and stuffed animals; an earlier batch contained about eight sessions. The first rig used two phones and a DJI action camera, with different intrinsics, rolling shutters, auto-exposure and autofocus. The cameras were not time-synchronised. The later rig recorded colour and depth with three Intel RealSense D415 cameras and measured the intrinsics and extrinsics manually as a reference, although the pipeline still infers poses; those sessions carry a metric depth reference, and the earlier batch has none.

5.2 Measuring Depth Quality Without Ground Truth

A depth error can only be measured against a sensor reference. PhysTwin's released sequences and the stricter self-collected sessions both carry sensor depth, but it is sparse and noisy relative to the dense predictions, and the looser-protocol videos have no reference at all. The evaluation therefore reports one relative comparison against the sensor (Section 5.3), covering a single sequence, and treats the rest qualitatively: flicker removal (Section 5.4), tracking (Section 5.5) and the end-to-end results. Each result is only a proxy, because smoothing can reduce flicker

while flattening or blurring geometry. There are no cross-view or warp-model scores, and no quantitative VDA or streaming-baseline comparison. End-to-end findings below are the author's own visual judgement. No other raters scored them, and no formal rating protocol was used. No loss-term ablation or temporal-span sweep was performed; their effects and weight sensitivity remain unmeasured.

5.3 Depth against the sensor reference

PhysTwin's released sequences carry RealSense depth, as do the stricter self-collected sessions, which were recorded with three RealSense D415 cameras capturing colour and depth together. The comparison below uses the released double-stretch sloth sequence: the raw inferred depth and the smoothed depth were each compared with the sensor reference over all 192 frames of the three cameras (`evaluation/compare_depth.py`). The comparison measures relative depth: Spearman rank correlation of the depth ordering, Pearson correlation of Sobel gradient magnitudes, the ordinal error (the fraction of sampled point pairs whose ordering is reversed), and the scale-invariant RMSE after a least-squares scale-and-shift alignment. The sensor depth is sparse and noisy around object boundaries, which limits all four measures.

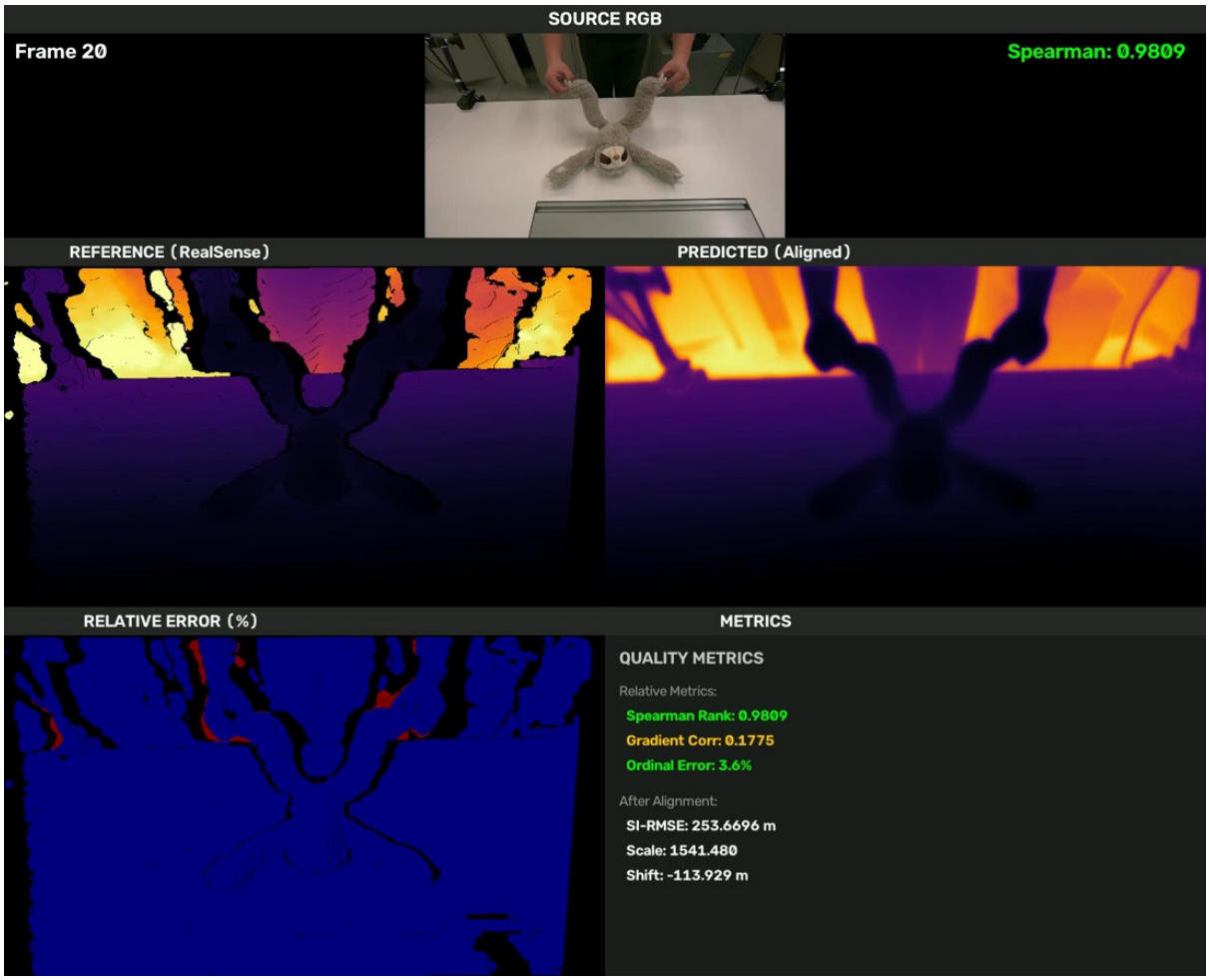


Figure 5.1: One frame (20 of 192) of the sloth sequence after smoothing: source RGB, RealSense reference depth, aligned smoothed depth, error map and per-frame metrics.

Table 5.1: Relative depth quality against the RealSense reference over 192 frames of the double-stretch sloth sequence, per camera, for the raw inferred depth (top) and the smoothed depth (bottom). SI-RMSE is in millimetres after scale-and-shift alignment.

Depth	Camera	Spearman ↑	Gradient corr. ↑	Ordinal error ↓	SI-RMSE ↓
Raw DA3	0	0.988	0.117	3.3%	346
	1	0.945	0.115	6.9%	666
	2	0.963	0.090	4.2%	1495
Smoothed	0	0.988	0.169	3.3%	343
	1	0.951	0.151	6.6%	664
	2	0.964	0.122	4.5%	1494

The depth ordering is nearly perfect for both variants and few point pairs are misordered. Smoothing leaves the ordering and the aligned error almost unchanged and raises the gradient correlation on every camera (0.107 to 0.147 on average), so the smoothed depth still agrees with the sensor reference. On camera 0 the per-frame ordinal error varies little (2.3–4.6%,

median 3.25%): most sampled pairs span large depth differences, which both variants order correctly, while the errors concentrate on near-tie pairs within the object and at its boundary. The gradient correlation is still modest: local depth structure remains weaker than the sensor's, which matches the inconsistent boundaries observed in Section 3.4. The measurement covers one sequence and compares the inferred depth with the sensor; it does not compare the four depth sources end to end.

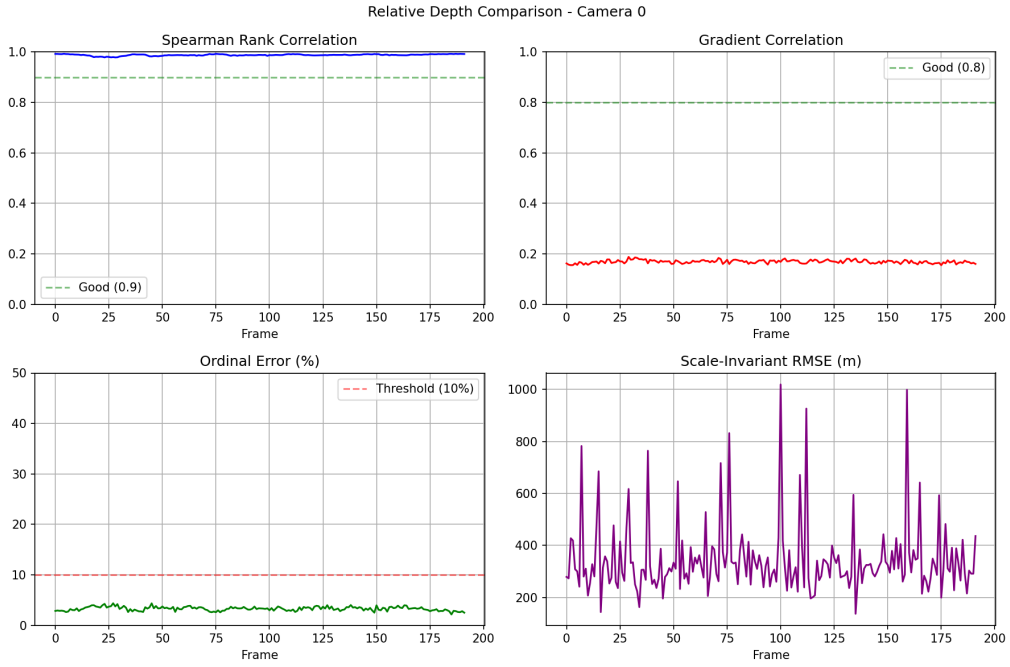


Figure 5.2: Per-frame relative depth metrics for the smoothed depth, camera 0: Spearman rank correlation, gradient correlation, ordinal error and SI-RMSE.

5.4 Flicker removal across sequences

Figures 5.3 and 5.4 show five consecutive frames of five sequences, raw DA3 above and smoothed below, with a shared colour scale per sequence. In the raw rows the depth of static surfaces shifts from frame to frame and the object–background boundary moves; the smoothed rows are stable frame to frame at the cost of softer boundaries. The smoothed rows come from the best saved model, `best_model_50_epochs_no_edge_blur`. Sequences include sloth, monkey, mat, rope and cloth; the monkey and mat rows use looser-protocol captures. Video recordings of the raw and smoothed depth are listed in Appendix A.2, and pipeline replays before and after smoothing in Appendix A.2.

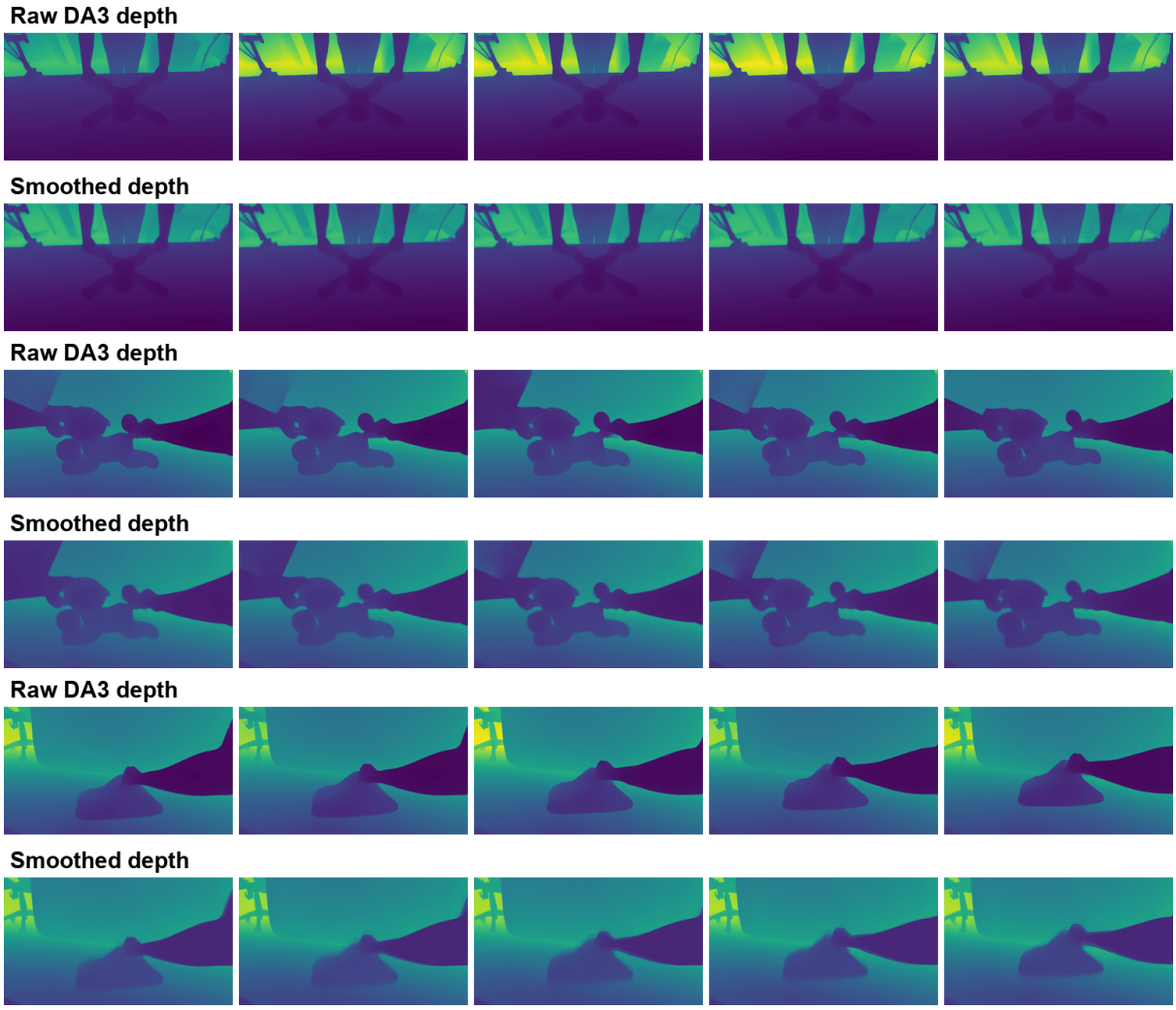


Figure 5.3: Raw DA3 (top) and smoothed depth (bottom) for five consecutive frames of each sequence: sloth stretch, monkey bend and mat lift.

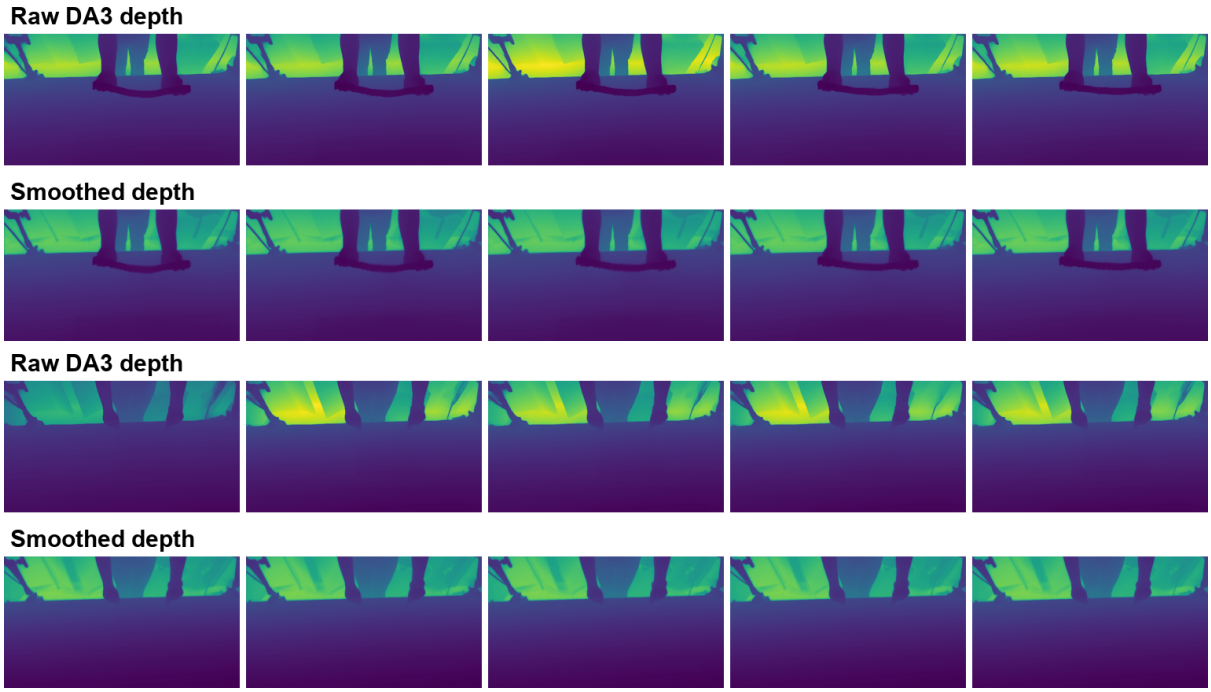


Figure 5.4: The same comparison for the rope and cloth sequences.

5.5 Effect on Tracking

The track comparisons ask how much the recovered 3D tracks change between pipeline variants. Each pair of runs reconstructs the same sequence from differently built input data: either the depth source differs (the original sensor depth and the new inferred depth), or only the temporal smoothing of the inferred depth differs. The comparison scripts did not isolate these factors reliably and reported differences that are not physically plausible: tens of centimetres of trajectory error between runs whose inputs differ only in the depth treatment. Those figures are therefore not reported here. The effect of the depth source itself is compared qualitatively in Section 5.7, and the cloth pair below is the clearest example of the effect of smoothing.

Figure 5.5 visualises that cloth pair (`single_lift_cloth_3_new` against `single_lift_cloth_3_smooth`), which differs only in whether the inferred depth was smoothed. The tracked object points of the two runs overlap almost completely, and the spread and velocity curves show that smoothing the inferred depth removes frame-to-frame jitter while the recovered motion is preserved.

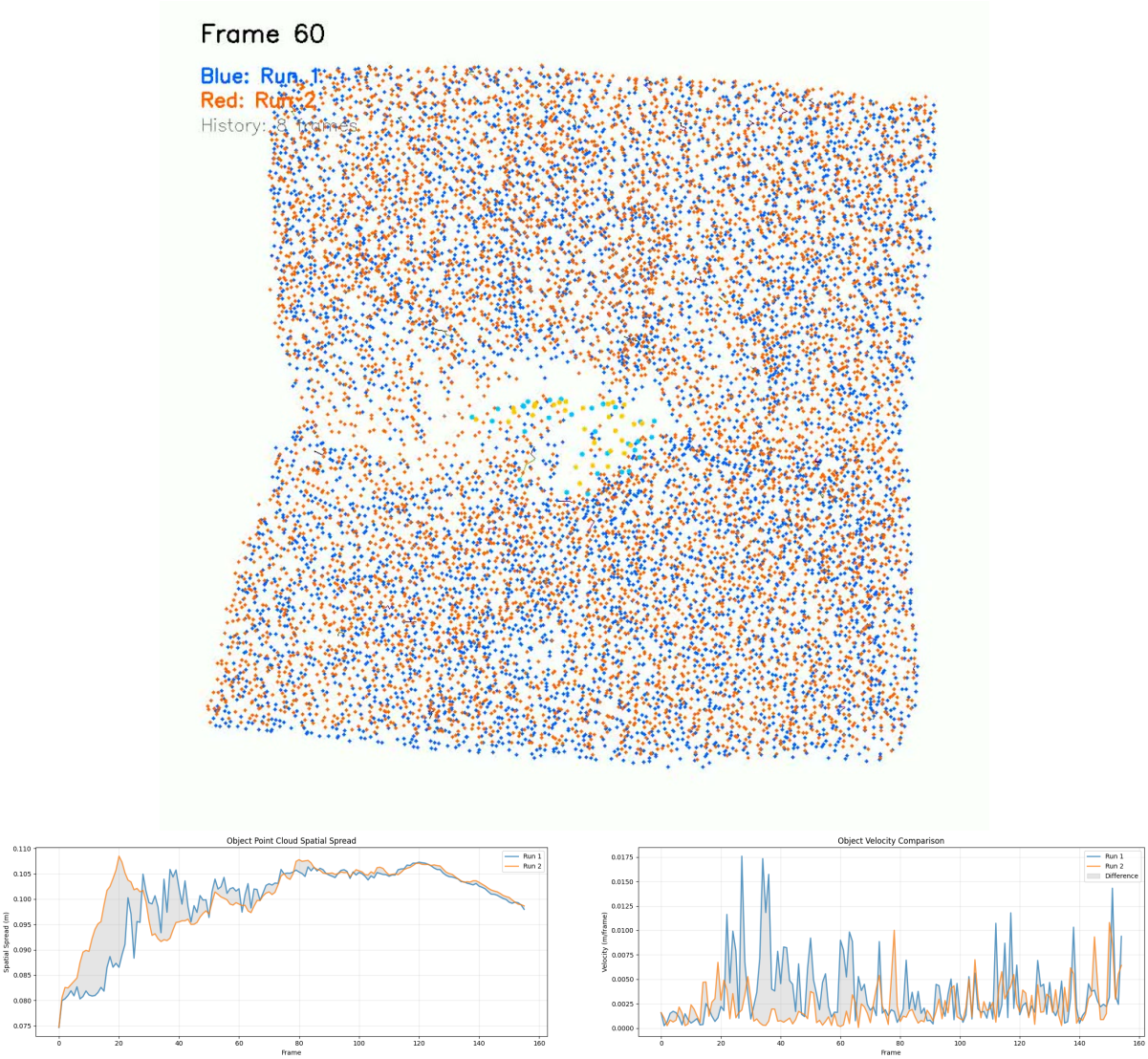


Figure 5.5: Tracking for the cloth pair (`single_lift_cloth_3_new` against `single_lift_cloth_3_smooth`). Top: tracked points of both runs overlaid. Bottom: object point-cloud spread and velocity; Run 1 unsmoothed, Run 2 smoothed.

5.6 End-to-End Results

DIPhysTwin completed on PhysTwin and self-collected data and produced interactive spring–mass warp models. On PhysTwin sequences, the author's visual inspection found the results broadly similar to the RGBD baseline. All runs use inferred poses, depth and ground alignment.

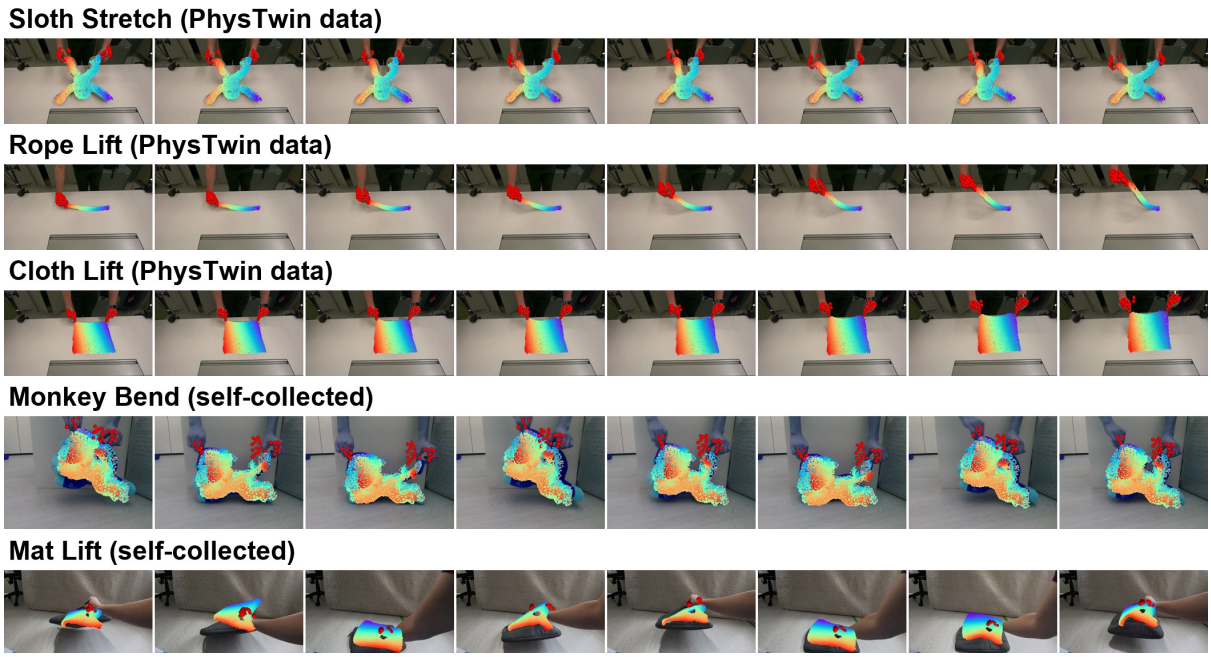


Figure 5.6: Warp-model results for five objects, eight frames sampled through each clip. Sloth, rope and cloth use PhysTwin data; monkey and mat are self-collected.

Recorded clips of the results are listed in Appendix A.

Figure 5.6 shows the trained warp models replaying the recorded interactions. The sloth, cloth and mat rows deform plausibly under the tracked controller motion; the rope row shows the thin-object case, where the model is driven by few object points. Monkey and mat are self-collected sessions and complete the pipeline with inferred pose and depth. Mat lift was recorded with looser constraints and still performs reasonably well.

5.7 Comparison with the original pipeline

Figure 5.7 compares the trained warp models of the original PhysTwin pipeline, which uses the sensor depth, with DIPhysTwin, which uses inferred depth, on the same sequences and at the same times; Figure 5.8 shows the self-collected monkey and mat sequences. For the monkey and mat pairs, re-captured sessions use recorded depth values, while retraining uses smoothed inferred depth. The sloth sequence is shown stage by stage in Appendix A.1.

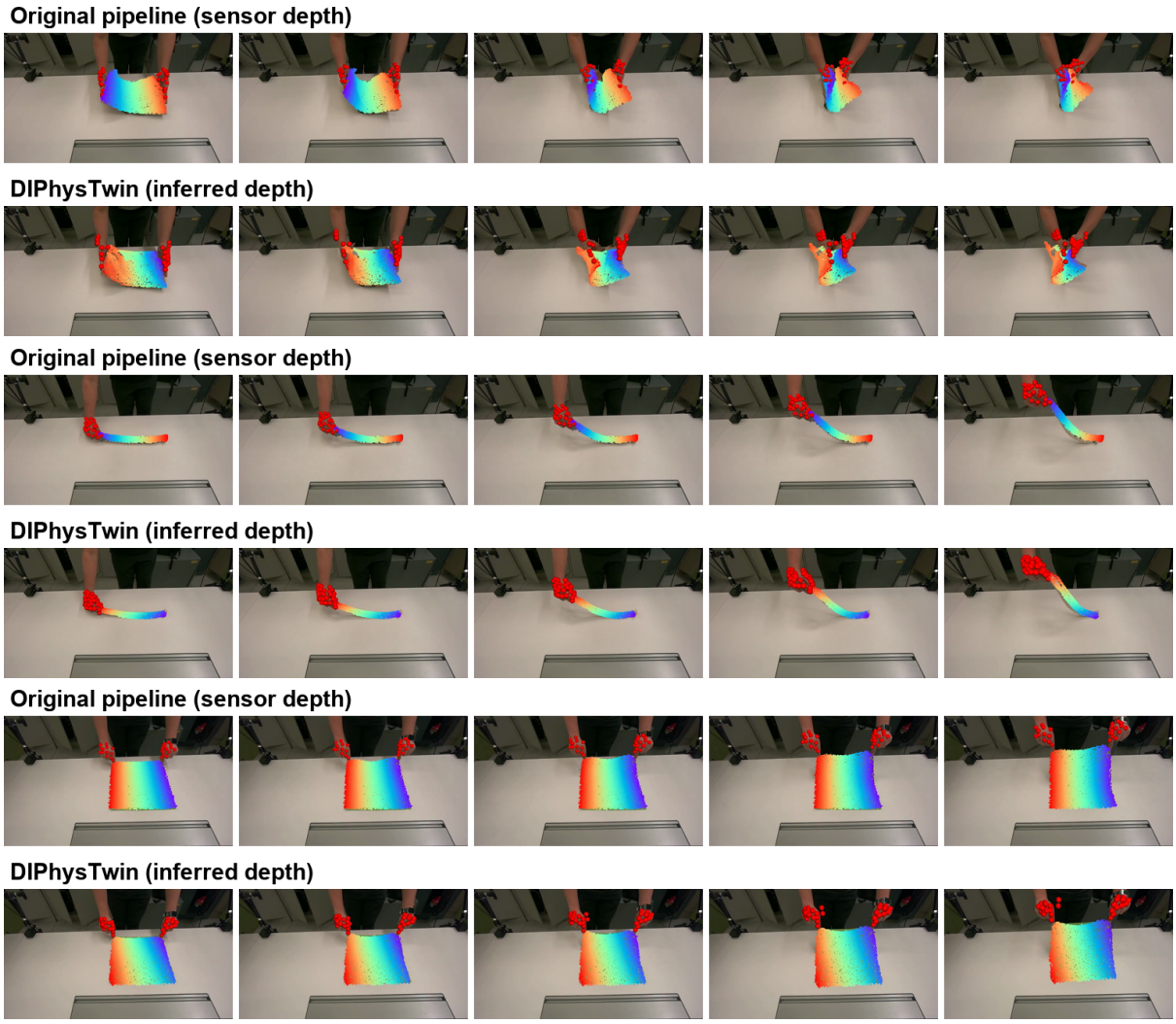


Figure 5.7: Original PhysTwin pipeline (top row of each object, sensor depth) against DIPhysTwin (bottom row, inferred depth) on the package, rope and cloth sequences, at matched times.

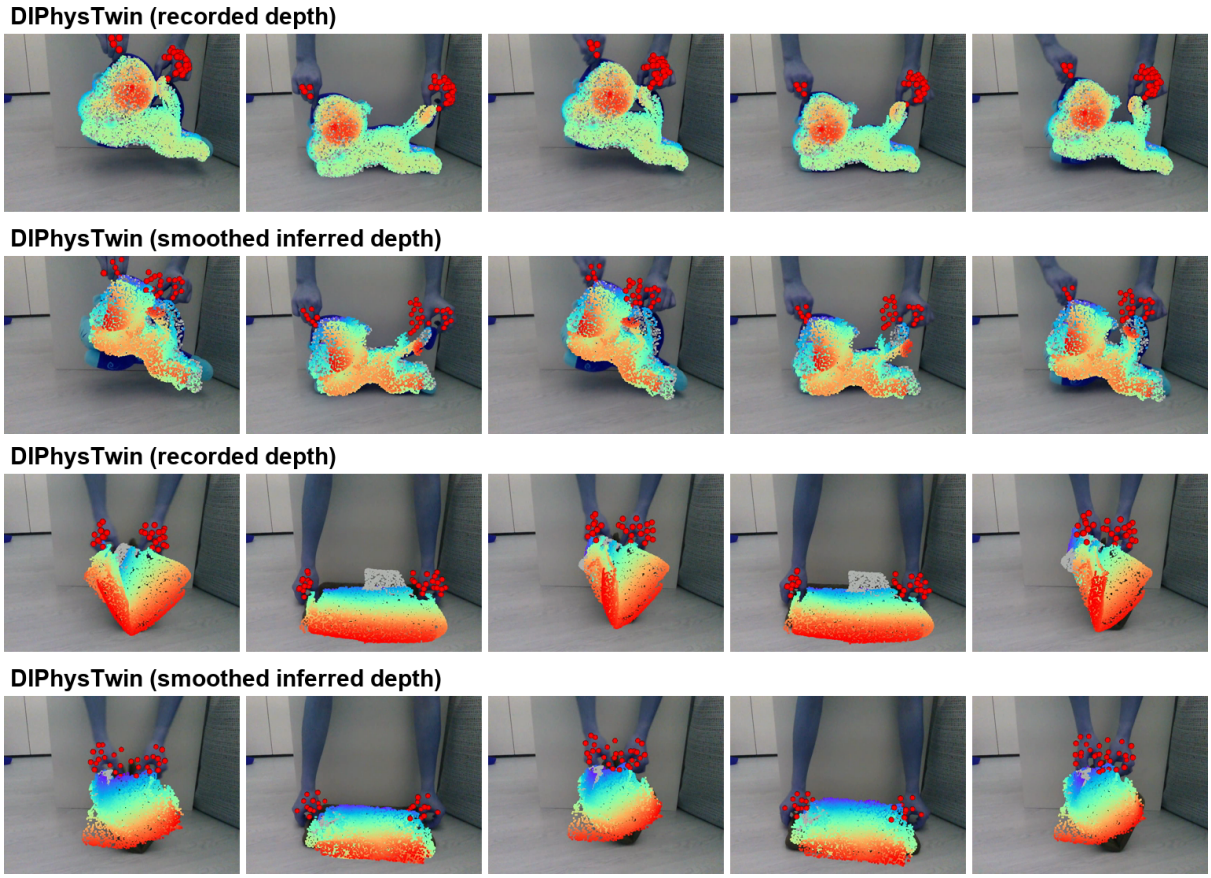


Figure 5.8: The monkey (stuffed animal) and mat sequences. Top row: DIPhysTwin runs with recorded depth; bottom row: runs with smoothed inferred depth.

The two rows are hard to compare cleanly. The inferred-depth runs still show jitter and 3D inconsistency, and these artefacts affect training overall. The two pipelines also differ in other respects, so the figure does not isolate how much of any difference comes from the depth source alone.

5.8 Findings and Failure Cases

Large, slow-moving objects with little occlusion reconstructed reliably. Heavy controller occlusion damaged segmentation, shape-prior generation and tracking. Small, fast-moving objects showed unstable depth and weaker 3D consistency; recovery also deteriorated when controller contact was lost. Spatial coherence weakened over time, especially in early unsynchronised captures, and unusual rotations reconstructed poorly. Soft objects without ground support required excessive stiffness before ground alignment was corrected; Appendix A.6 shows the rope sequence before and after the correction.

Table 5.2: The author's qualitative assessment of capture conditions and outcome across the self-collected sessions. Outcomes are not counted per session.

Object	Scale	Motion speed	Controller occlusion	Outcome
Mat or foam pad	Large	Slow	Minimal	Good
Stuffed animal	Moderate	Slow	Minimal	Good
Cloth	Large	Fast	Significant	Partial
Bottle	Small	Fast	Significant	Poor
Mug	Small	Fast	Significant	Poor
Rope	Large, thin	Slow	None (no ground support)	Partial

5.9 Negative Results

Different tools and methodologies were also tested but did not improve the pipeline. The following list is not exhaustive, but it covers the main alternatives that were tried and abandoned.

Replacing DA3 with π^3 failed to preserve alignment with known camera poses (Yifan Wang et al. 2026). MVTracker performed worse than 2D tracking, while SpatialTrackerV2 produced frequent 3D artefacts (Rajič et al. 2025; Xiao et al. 2025) (Appendix A.5). Trellis and the current CoTracker version gave no visible improvement (Xiang et al. 2025; Karaev et al. 2024). Inference without chunk stitching was also tested; its recordings are listed in Appendix A.4.

Chapter 6

Discussion, Limitations and Future Work

6.1 Summary and Diagnosis

DIPhysTwin adapts PhysTwin to multi-view RGB video: camera poses, depth and ground alignment are inferred, then handed to its reconstruction and warp-model stages. The new component is a residual depth smoother, supervised by Video Depth Anything (VDA) as a motion oracle and trained without ground-truth depth (Jiang et al. 2025; Chen et al. 2025).

The pipeline is still 2D-first: segmentation and CoTracker run on RGB, and depth lifts their output into 3D (Karaev et al. 2024). Depth errors therefore reach the tracks, the shape prior, the reconstruction and the warp model at once. No tested depth model was good at both jobs. DUSt3R, MASt3R and DA3 give geometry but flicker, while VDA is smooth but monocular and geometrically weak (S. Wang et al. 2024; Leroy et al. 2024; Lin et al. 2026; Chen et al. 2025). The smoother takes geometry from the first group and motion from the second, at the cost of some blur (Chapter 5).

Two alternatives were considered and dropped. Replacing the weak parts of the PhysTwin pipeline with a multi-pose streaming depth model and multi-view tracking would require a new model (S. Wang et al. 2024; Leroy et al. 2024; Karaev et al. 2024). Rewriting around 4D Gaussian splatting would remove explicit poses but move segmentation and tracking into 3D (Wu et al. 2024; Kerbl et al. 2023).

6.2 Limitations

The depth stage is not truly streaming. Duplicating fixed views as a dynamic sequence helps only over short windows; larger chunks make the Umeyama alignment singular (Umeyama 1991). The three cameras also differ in intrinsics and distortion, use rolling shutters and automatic exposure and focus, and were never time-synchronised; software only aligned their start times. The later rig used measured intrinsics and extrinsics only as a reference, and may still be slightly fisheye. These conditions explain the observed symptoms: flicker, spatial drift within a clip, and failures on fast motion and unusual rotations.

The environment is static. The floor collider fixed the over-stiff behaviour for supported objects, but a rope has little ground support and still needs unrealistic stiffness. The smoother blurs depth, and the current evaluation cannot say whether it removes flicker without also removing fine geometry.

The depth comparison covers one sequence, the remaining evidence is qualitative, and the loss weights are untuned. The findings on object size, speed and occlusion are observed trends from a small number of sessions.

6.3 Future Work

The depth stage needs true multi-view streaming. A model that accepts several camera poses would replace the chunk-overlap workaround and its unstable alignment (Umeyama 1991). The same model should be both spatially consistent and temporally smooth, and the shape prior needs to handle non-watertight meshes (Sorkine and Alexa 2007).

The capture should be repeated with synchronisation cues, locked exposure and focus, and measured intrinsics, allowing for fisheye distortion. Tracking needs to survive occlusion and partially visible objects; MVTracker and direct 3D tracking are the next candidates (Rajič et al. 2025; Xiao et al. 2025). Environment interaction should move from static planes to moving rigid bodies.

The smoother needs loss-term and span ablations, multi-view evaluation, and comparison against reference geometry where any exists. After that the pipeline should be cleaned up and released.

6.4 Conclusion

DIPhysTwin turns multi-view RGB video into an interactive digital twin, and the smoother reduces temporal flicker without ground-truth depth. Results are strongest for large, slow-moving objects under controlled capture. Depth coherence, occlusion, calibration and tracking remain the main limitations.

References

AI, Meta (2025). *SAM 3: Segment Anything with Concepts*. arXiv:2511.16719.

Chen, Sili, Guo, Hengkai, Zhu, Shengnan, Zhang, Feihu, Huang, Zilong, Feng, Jiashi, and Kang, Bingyi (2025). "Video Depth Anything: Consistent Depth Estimation for Super-Long Videos". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Jiang, Hanxiao, Hsu, Hao-Yu, Zhang, Kaifeng, Yu, Hsin-Ni, Wang, Shenlong, and Li, Yunzhu (2025). *PhysTwin: Physics-Informed Reconstruction and Simulation of Deformable Objects from Videos*. `https://jianghanxiao.github.io/phystwin-web/`. ICCV 2025.

Karaev, Nikita, Rocco, Ignacio, Graham, Benjamin, Neverova, Natalia, and Vedaldi, Andrea (2024). "CoTracker: It is Better to Track Together". In: *European Conference on Computer Vision (ECCV)*.

Kerbl, Bernhard, Kopanas, Georgios, Leimkühler, Thomas, and Drettakis, George (2023). "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4.

Kirillov, Alexander, Mintun, Eric, Ravi, Nikhila, Mao, Hanzi, Rolland, Chloe, Gustafson, Laura, Xiao, Tete, Whitehead, Spencer, Berg, Alexander C., Lo, Wan-Yen, Dollár, Piotr, and Girshick, Ross (2023). "Segment Anything". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Leroy, Vincent, Cabon, Yohann, and Revaud, Jérôme (2024). "Grounding Image Matching in 3D with MASt3R". In: *European Conference on Computer Vision (ECCV)*.

Li, Pengzhi, Ding, Yikang, Li, Linge, Guan, Jingwei, and Li, Zhiheng (2023). "Towards Practical Consistent Video Depth Estimation". In: *Proceedings of the 2023 ACM International Conference on Multimedia Retrieval (ICMR)*, pp. 388–397. DOI: 10.1145/3591106.3592264.

Lin, Haotong, Chen, Sili, Liew, Jun Hao, Chen, Donny Y., Li, Zhenyu, Zhao, Yang, Peng, Sida, Guo, Hengkai, Zhou, Xiaowei, Shi, Guang, Feng, Jiashi, and Kang, Bingyi (2026). *Depth Anything 3: Recovering the Visual Space from Any Views*. ICLR 2026. arXiv:2511.10647.

Macklin, Miles (Mar. 2022). *Warp: A High-performance Python Framework for GPU Simulation and Graphics*. NVIDIA GPU Technology Conference (GTC). URL: https://github.com/NVIDIA/warp.

Rajič, Frano, Xu, Haofei, Mihajlovic, Marko, Li, Siyuan, Demir, Irem, Gündöğdu, Emircan, Ke, Lei, Prokudin, Sergey, Pollefeys, Marc, and Tang, Siyu (2025). "Multi-View 3D Point Tracking". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Ren, Tianhe, Liu, Shilong, Zeng, Ailing, Lin, Jing, Li, Kunchang, Cao, He, Chen, Jiayu, Huang, Xinyu, Chen, Yukang, Yan, Feng, Zeng, Zhaoyang, Zhang, Hao, Li, Feng, Yang, Jie, Li, Hongyang, Jiang, Qing, and Zhang, Lei (2024). *Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks*. arXiv:2401.14159.

Sobko, Ivan, Riemenschneider, Hayko, Gross, Markus, and Schroers, Christopher (2026). *StableDPT: Temporal Stable Monocular Video Depth Estimation*. arXiv:2601.02793.

Sorkine, Olga and Alexa, Marc (2007). "As-Rigid-As-Possible Surface Modeling". In: *Eurographics Symposium on Geometry Processing (SGP)*.

Umeyama, Shinji (1991). "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.4, pp. 376–380.

Wang, Shuzhe, Leroy, Vincent, Cabon, Yohann, Chidlovskii, Boris, and Revaud, Jérôme (2024). "DUSt3R: Geometric 3D Vision Made Easy". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Wang, Yifan, Zhou, Jianjun, Zhu, Haoyi, Chang, Wenzheng, Zhou, Yang, Li, Zizun, Chen, Junyi, Pang, Jiangmiao, Shen, Chunhua, and He, Tong (2026). π^3*: Permutation-Equivariant Visual Geometry Learning*. ICLR 2026. arXiv:2507.13347.

Wang, Yiran, Shi, Min, Li, Jiaqi, Hong, Chaoyi, Huang, Zihao, Peng, Juewen, Cao, Zhiguo, Zhang, Jianming, Xian, Ke, and Lin, Guosheng (2025). "NVDS+: Towards Efficient and Versatile Neural Stabilizer for Video Depth Estimation". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47.1, pp. 583–600.

Wang, Yiran, Shi, Min, Li, Jiaqi, Huang, Zihao, Cao, Zhiguo, Zhang, Jianming, Xian, Ke, and Lin, Guosheng (2023). "Neural Video Depth Stabilizer". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Wu, Guanjun, Yi, Taoran, Fang, Jiemin, Xie, Lingxi, Zhang, Xiaopeng, Wei, Wei, Liu, Wenyu, Tian, Qi, and Wang, Xinggang (2024). "4D Gaussian Splatting for Real-Time Dynamic Scene Rendering". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Xiang, Jianfeng, Lv, Zelong, Xu, Sicheng, Deng, Yu, Wang, Ruicheng, Zhang, Bowen, Chen, Dong, Tong, Xin, and Yang, Jiaolong (2025). "Structured 3D Latents for Scalable and Versatile 3D Generation". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Xiao, Yuxi, Wang, Jianyuan, Xue, Nan, Karaev, Nikita, Makarov, Iurii, Kang, Bingyi, Zhu, Xing, Bao, Hujun, Shen, Yujun, and Zhou, Xiaowei (2025). "SpatialTrackerV2: Advancing 3D Point Tracking with Explicit Camera Motion". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Appendix A

Extended Results and Result Videos

The recordings below accompany this report. They are grouped by what they compare: within each comparison table one row shows the same scene under two or more pipeline variants, so the effect of a single change can be seen directly. Scene names describe the captured interaction (for example "cloth, single lift" is a cloth lifted from one point). Recording links are given in shortened form.

A.1 Improvement Chain on the Sloth Sequence

Table A.1 shows the improvement chain on the sloth sequence: the original pipeline (sensor reference depth), the pipeline with DA3 inferred depth, and the pipeline with smoothed DA3 depth. Each stage replaces one component of the previous one. The measurements in Chapter 5 quantify the last step: for this sequence the smoothed depth keeps the agreement with the sensor reference (Spearman 0.988, ordinal error 3.3% on camera 0) with a higher gradient correlation than raw DA3 (0.169 against 0.117; Table 5.1).

Table A.1: Sloth sequence: the three pipeline stages.

Stage	Recording
1. Original pipeline (reference depth)	`youtu.be/UyQw7Hx25D8`
2. DA3 inferred depth with streaming	`youtu.be/WldxhW5DKHs`
3. Smoothed DA3 inferred depth	`youtu.be/xjoEsQoWlXw`

A.2 Effect of the Temporal Depth Smoother

Raw DA3 depth against smoothed depth. Each row of Table A.2 shows one scene twice: the raw DA3 depth prediction and the same frames after temporal smoothing. The raw recordings show the frame-to-frame flicker that motivates the smoother (Chapter 4): the depth of static surfaces shifts between consecutive frames and the object–background boundary moves.

Table A.2: Raw DA3 depth against smoothed depth.

Scene	Raw DA3 depth	Smoothed depth
Cloth, single lift	youtu.be/c_K8Iw4v8GM	youtu.be/Oix-bPoGo1Y
Zebra, double lift	youtu.be/22x_s_dwmJA	youtu.be/jrlRHbfc8wo
Mat, single lift	youtu.be/CfmDhliW4e4	youtu.be/eaA4PYEcdmU

Pipeline replay before and after smoothing. Table A.3 shows the pipeline replay of the same two scenes before and after the inferred depth was smoothed, at matched times.

Table A.3: Pipeline replay before and after depth smoothing.

Scene	Before smoothing	After smoothing
Cloth, single lift	youtu.be/oMyWfrkThjA	youtu.be/sFhpqg2umsA
Sloth, double stretch	youtu.be/Pyi-6zr_SVY	youtu.be/DJOm86ircG4

A.3 Depth-Source Variants

Table A.4 lists further depth sources used for the same inference comparison. The reference and DA3 rows repeat the first two stages of Table A.1 so that the set is self-contained.

Table A.4: Inference with alternative depth sources.

Depth source	Recording	Notes
Reference (sensor depth)	youtu.be/UyQw7Hx25D8	original pipeline
DA3 inferred depth with streaming	youtu.be/Wldxhw5DKHs	adopted depth source
VDA	youtu.be/GUog6wd8YiI	video model oracle
Without streaming	youtu.be/NbUOTzVzMGM	without chunk stitching

A.4 Streaming Inference

Table A.5 compares depth inference with and without chunk stitching (streaming), for the final data and the final matching visualisations, with a reference recording of the final data.

Table A.5: Streaming inference against non-streaming inference.

Visualisation	Without streaming	With streaming	Reference
Final data	youtu.be/wtfQiYJZQfQ	youtu.be/b3FfEog8buc	youtu.be/bH6H2vx3Q4k
Final matching	youtu.be/m4g420lpaH4	youtu.be/M2VwlY6B9j4	—

A.5 Tracking Comparison with SpatialTrackerV2

Table A.6 compares the tracked result of the same monkey sequence with the 2D-tracking-based pipeline and with SpatialTrackerV2 (Xiao et al. 2025).

Table A.6: Monkey sequence: DIPhysTwin against SpatialTrackerV2.

Variant	Recording	Notes
DIPhysTwin	youtu.be/8WwHEsOErsI	spatial separation on fast movement
SpatialTrackerV2	youtu.be/DijnnXjS_Xc	frequent 3D artefacts

A.6 Environment Interaction: Rope with and without Ground Alignment

Table A.7 shows the rope sequence before and after the ground-plane alignment was added (Chapter 3), together with the reference recording of the same interaction.

Table A.7: Rope sequence: stiff result, reference interaction, and recovered ground contact.

Variant	Recording	Notes
Without ground alignment	youtu.be/I59sUpi_Rvk	rope overly stiff
Reference interaction	youtu.be/fhun0051Rps	rope rests on the ground
With ground alignment	youtu.be/zXyEl8aMBYk	ground contact recovered

A.7 Individual Demonstrations

Single recordings without a paired comparison, grouped by type.

Table A.8: Individual demonstration recordings.

Recording	Scene	Notes
Tracking results		
youtu.be/94h6tWvJEwE	Mat, double bend	tracking reconstruction
youtu.be/ICQz9aUabAo	Mat, single lift	tracking reconstruction
Pipeline demonstrations, PhysTwin data		
youtu.be/nsndv85sxcU	Sloth, double stretch	
youtu.be/t97kz2DZZxk	Zebra, double lift	
youtu.be/wVsCuG4sxWI	Cloth, double hand lift	
youtu.be/FUNXNBtAupQ	Cloth, single lift	degraded result
youtu.be/I59sUpi_Rvk	Rope, single lift	see Table A.7
Pipeline demonstrations, self-collected data		
youtu.be/U8ICkg25_4U	Mat, double bend	
youtu.be/rtjAy8hZE8k	Mat, single lift	
youtu.be/RTn-0Zg8wAA	Water bottle, single lift	poor result
youtu.be/SbAI6K0lMDk	T-shirt, single lift	
youtu.be/svUzZCJdgDk	Monkey, double bend	
youtu.be/yZ0TzMy9XA0	Package, single lift	
New-data captures, three camera corners		
youtu.be/sFbwJNk7rUs	Mat, single lift	camera corner 0
youtu.be/LdueFxEb7H8	Mat, single lift	camera corner 1
youtu.be/ZcDswgNRI40	Mat, single lift	camera corner 2
Issue and failure recordings		
youtu.be/e6UjWpdsnjg	Mat, double bend	object split during interaction
youtu.be/8WwHEsOErsI	Monkey, double bend	see Table A.6
youtu.be/4lE1MX_xUnM	Rope, single lift	depth flicker propagating into 3D tracks
Further recordings		
youtu.be/TVVKJ1hBin8	Sloth, double stretch	interactive playground session
youtu.be/cywRH67oSGY	Sloth, double stretch	depth-quality comparison with metrics
youtu.be/R-0QuXLqUTQ	Mat, double bend	early result, poor
youtu.be/44K07VyE1iI	Mat, single lift	early result

Appendix B

Hyper-parameters and Loss Weights

This appendix records the settings needed to reproduce the temporal depth smoother. The values are the training configurations saved with the checkpoints.

Table B.1: Training and inference hyper-parameters of the reported smoother run.

Setting	Value
Optimiser	Adam; weight decay 10^{-4}
Learning rate	3×10^{-3}
Schedule	ReduceLROnPlateau; factor 0.5, patience 5
Batch size	2
Epochs	50; checkpoints saved for epochs 0–49
Precision	CUDA autocast with bfloat16
Hardware	NVIDIA GeForce RTX 3060
Inference chunk length	16 frames per chunk, matching the training window; chunks overlap and are blended with a weighted window

Table B.2: Loss weights of the best saved smoother runs; the first row is the run behind the reported results. λ_{edge} and λ_{grad} are the edge-sharpness and spatial-gradient terms of Section 4.4.

Run	λ_{fid}	λ_{tgm}	λ_{tv}	λ_{edge}	λ_{grad}
`best_model`	1.2	0.5	0.5	0.3	0
`best_model_50_epochs`	1.2	0.5	0.5	0.3	0
`best_model_50_epochs_no_edge_blur`	1.3	0.8	0.8	0.05	0.04
`best_model_new_data`	1.3	0.8	0.8	0.05	0.05

The temporal spans follow the code defaults used by all runs: TGM $k = 1, 2, 3$ with weights $1, 0.25, 0.07$, and TV $k = 1, 2$ with weights $1, 0.25$.

Appendix C

Source Code

The implementation is released at `https://github.com/Hanson-Sun/PhysTwin/tree/depth-inference-phystwin`.